



VIT[®]
BHOPAL

Project Report

VIT BHOPAL UNIVERSITY

Sehore, Madhya Pradesh

Submitted to: Dheresh Soni

Submitted by: Lalan Kumar Gupta

Registration no: 25BAI11005

Couse Title: Programming In Java

Course code: CSE2006

Program: B.tech

Session : 2026-27

Branch : SCAI

Date: 18 Sep 2026

Git Hub Link: <https://github.com/lalan25bai11005/Student-Record-Management-System>

1. INTRODUCTION

Managing student information is an important activity in an educational environment. Student details such as name, age, department, semester, contact information, marks, and attendance need to be maintained properly so that they can be accessed and updated when required.

In a manual system, maintaining such information can become inconvenient, especially when the number of records increases. Searching for a particular student, updating information, calculating academic performance, and maintaining records separately can require additional effort.

To address this problem at a basic level, the Student Record Management System was developed using Java.

The application provides a simple menu-driven console interface through which the user can manage student information and academic performance. The system also provides file-handling functionality so that records can be stored in a text file.

The project focuses on applying the concepts learned in Core Java and Object-Oriented Programming rather than developing a large-scale commercial system.

2. PROBLEM STATEMENT

Maintaining student records manually can become difficult when the number of students increases.

Searching for a particular student, modifying existing information, calculating academic performance, and maintaining records in an organized manner can be time-consuming.

The problem addressed by this project is to develop a simple Java-based system that can:

- Store student information in an organized manner.
- Add new student records.
- Search students using Student ID.
- Update existing student information.
- Delete student records.
- Store academic marks and attendance.
- Calculate average marks and grades.
- Display academic performance.
- Save records to a text file.
- Handle invalid user inputs appropriately.

The Student Record Management System provides these functionalities through a console-based Java application.

3. OBJECTIVES

The major objectives of this project are:

1. To develop a functional student record management application using Java.
2. To apply Object-Oriented Programming concepts in a practical application.
3. To implement basic CRUD operations for student records.
4. To use ArrayList for managing multiple student objects.
5. To use arrays for storing subject marks.
6. To calculate student average marks and grades automatically.
7. To maintain attendance information.
8. To implement exception handling for invalid input.
9. To demonstrate Java file handling using FileWriter, FileReader, and BufferedReader.
10. To create a modular application by dividing responsibilities among multiple Java classes.

4. SCOPE OF THE PROJECT

The project focuses on managing basic student and academic information through a console-based application.

4.1 Features Included in the Scope

The current system includes:

- Student registration
- Displaying student records
- Searching students
- Updating student information
- Deleting student records
- Academic marks management
- Attendance management
- Average calculation
- Grade calculation
- Performance classification

- File-based record storage
- Input validation
- Exception handling

4.2 Features Outside the Current Scope

The current version does not include:

- Graphical User Interface
- Database integration
- Online access
- User authentication
- Cloud storage
- Network communication

These features can be considered for future versions.

5. TARGET USERS

The system is mainly intended for:

- Students learning Java programming.
- Faculty members evaluating Java projects.
- Beginners learning Object-Oriented Programming.
- Small academic demonstrations.
- Users requiring a simple local student record system.

The project is particularly suitable for academic use because it connects Java programming concepts with a practical student management problem.

6. FUNCTIONAL REQUIREMENTS

The project contains four major functional areas.

6.1 Student Management

The Student Management module is responsible for maintaining student information.

The user can:

- Add a new student.
- Display all students.

- Search for a student using Student ID.
- Update student information.
- Delete a student.

The system also prevents duplicate Student IDs.

6.2 Academic Performance

The Academic Performance module allows marks to be entered for the following subjects:

- Java
- Operating System
- Digital Logic
- Data Structures
- Computer Organization

The system calculates the average and automatically assigns a grade and performance level.

6.3 Attendance Management

The system accepts attendance as a percentage.

If the attendance is below 75%, a warning is displayed.

For example:

Attendance: 68.00%

Warning: Attendance is below 75%!

If attendance is 75% or above, the system displays:

Attendance: Satisfactory

6.4 File Management

The File Management module provides functionality to save and read student records.

The records are stored in a text file named:

student_records.txt

The user can save records through the main menu and later read the saved file contents.

7. NON-FUNCTIONAL REQUIREMENTS

7.1 Usability

The system uses a simple menu-driven console interface. The available operations are clearly displayed so that users can easily understand the system.

7.2 Reliability

The application validates important inputs and handles common errors using exception handling.

7.3 Maintainability

The project is divided into separate Java classes. Each class has a specific responsibility, making the source code easier to understand and modify.

7.4 Error Handling

The application handles invalid numerical input, duplicate Student IDs, invalid marks, invalid attendance, and non-existing student searches.

7.5 Resource Efficiency

The application uses simple data structures and file operations. It does not require a database server or external infrastructure.

7.6 Performance

For a small academic dataset, the application provides sufficiently fast operations for interactive console usage.

8. SYSTEM ARCHITECTURE

The application follows a simple modular architecture.

Main.java acts as the entry point of the application and provides the main menu.

StudentManager.java manages student-related operations.

Student.java represents an individual student.

Performance.java manages academic performance and attendance.

FileManager.java handles file-based storage.

The overall architecture can be represented as:

Main

↓

StudentManager

↓
Student and Performance
↓
FileManager
↓
student_records.txt

The modular structure ensures that each class performs a clearly defined task.

9. PROJECT WORKFLOW

The general workflow of the application is:

Start the application

↓
Display Main Menu

↓
User selects an operation

↓
Add Student / Display Students / Search Student / Update Student / Delete Student / Manage
Performance / Save Records / Read Records

↓
Perform the selected operation

↓
Display the result

↓
Return to Main Menu

↓
User selects Exit

↓
End the application

The application continues running until the user selects the Exit option.

10. USE CASE DIAGRAM

Actor

User

Main Use Cases

The user interacts with the system through the following operations:

- Add Student
- Display Students
- Search Student
- Update Student
- Delete Student
- Add or Update Performance
- Display Performance
- Save Records
- Read Records
- Exit System

[Insert Use Case Diagram Here]

Figure 1: Use Case Diagram of Student Record Management System

11. CLASS DIAGRAM

The project consists of five Java classes:

1. Main
2. Student
3. StudentManager
4. Performance
5. FileManager

The relationships between the classes can be represented as follows:

Main



StudentManager



Student



Performance

StudentManager



FileManager

[Insert Class Diagram Here]

Figure 2: Class Diagram of Student Record Management System

12. SEQUENCE DIAGRAM

A typical academic performance operation follows this sequence:

1. The user selects “Add/Update Performance”.
2. Main passes the request to StudentManager.
3. StudentManager searches for the student using Student ID.
4. A Performance object is created.
5. The user enters marks.
6. The user enters attendance.
7. Performance calculates the average and grade.
8. The Performance object is associated with the Student object.
9. The system displays a success message.

[Insert Sequence Diagram Here]

Figure 3: Sequence Diagram for Adding Student Performance

13. STORAGE DESIGN

The current project does not use a database.

Instead, student records are stored in a text file named:

student_records.txt

The file contains information such as:

Student ID

Name

Age

Gender

Department

Semester

Email

Phone

Subject Marks

Average

Attendance

Grade

Performance Level

The use of a text file allows the project to demonstrate Java file handling without requiring database software.

An ER diagram is therefore not applicable to the current implementation.

14. DESIGN DECISIONS AND RATIONALE

14.1 Choice of Java

Java was selected because the project is intended to demonstrate concepts from the Java Programming course. Java provides built-in support for Object-Oriented Programming, collections, exception handling, and file I/O.

14.2 Console-Based Interface

The application was designed as a console-based system to keep the focus on Java programming logic rather than graphical interface development.

This also makes the application lightweight and easy to compile and execute.

14.3 Use of ArrayList

ArrayList is used to maintain student records because the number of students can change during program execution.

It provides convenient operations such as adding and removing student objects.

14.4 Use of Separate Classes

The project is divided into five classes instead of placing all functionality inside one class.

This improves code organization and makes individual components easier to understand and maintain.

14.5 File-Based Storage

Since database integration was not included in the project scope, a text file was selected as a simple storage mechanism.

This also demonstrates Java file-handling concepts.

15. IMPLEMENTATION DETAILS

15.1 Main.java

Main.java is the entry point of the application.

Its main responsibilities are:

- Starting the application.
- Creating the Scanner object.
- Creating the StudentManager object.
- Displaying the main menu.
- Reading the user's choice.
- Calling the appropriate StudentManager method.
- Exiting the application when requested.

A switch statement is used to process menu choices.

15.2 Student.java

Student.java represents an individual student.

It contains information such as:

- Student ID
- Name
- Age

- Gender
- Department
- Semester
- Email
- Phone
- Performance

The class uses private variables and public getter and setter methods.

This demonstrates the concept of encapsulation.

15.3 StudentManager.java

StudentManager.java manages the collection of student objects.

Its major operations include:

- addStudent()
- displayAllStudents()
- searchStudent()
- updateStudent()
- deleteStudent()
- addPerformance()
- displayPerformance()

Student records are maintained using ArrayList<Student>.

15.4 Performance.java

Performance.java manages academic performance.

It stores marks using an integer array and maintains attendance, average, grade, and performance level.

The average is calculated using:

Average = Total Marks / Number of Subjects

The grade is then determined based on the calculated average.

15.5 FileManager.java

FileManager.java manages file operations.

The project uses:

- FileWriter
- FileReader
- BufferedReader

FileWriter is used to save records, while BufferedReader and FileReader are used to read the stored records.

Try-with-resources is used so that file resources are properly closed after the operation.

16. EXCEPTION HANDLING

Exception handling is an important part of the project because users may enter unexpected values.

For example, if the program expects a number and the user enters text, NumberFormatException can occur.

The application handles such situations using try-catch blocks.

The system also handles:

- Invalid Student ID
- Invalid age
- Invalid semester
- Invalid marks
- Invalid attendance
- File reading/writing errors

Instead of terminating unexpectedly, the application displays an appropriate error message and allows the user to continue.

17. INPUT VALIDATION

Input validation is implemented at different stages of the application.

Student ID

Student IDs must be unique.

Name

The student name cannot be empty.

Department

The department cannot be empty.

Age

Age must be greater than zero.

Semester

Semester must be a positive value.

Marks

Marks must be between 0 and 100.

Attendance

Attendance must be between 0 and 100 percent.

These validations help prevent invalid data from being stored in the application.

18. TESTING APPROACH

The application was tested using different functional and validation scenarios.

Test Case 1: Add Student

Input:

Student ID: 101

Name: Rahul

Age: 20

Department: CSE

Semester: 3

Expected Result:

Student added successfully.

Result:

Passed

Test Case 2: Duplicate Student ID

Input:

Student ID: 101 when ID 101 already exists.

Expected Result:

Student ID already exists.

Result:

Passed

Test Case 3: Search Existing Student

Input:

Student ID: 101

Expected Result:

Student information is displayed.

Result:

Passed

Test Case 4: Search Non-Existing Student

Input:

Student ID: 999

Expected Result:

Student not found.

Result:

Passed

Test Case 5: Invalid Marks

Input:

Marks: 120

Expected Result:

Marks must be between 0 and 100.

Result:

Passed

Test Case 6: Invalid Attendance

Input:

Attendance: 105

Expected Result:

Attendance must be between 0 and 100.

Result:

Passed

Test Case 7: Low Attendance

Input:

Attendance: 68%

Expected Result:

Attendance warning is displayed.

Result:

Passed

Test Case 8: Save Records

Action:

Select Save Records to File.

Expected Result:

Records are successfully saved in student_records.txt.

Result:

Passed

Test Case 9: Read Records

Action:

Select Read Records from File.

Expected Result:

Previously saved records are displayed.

Result:

Passed

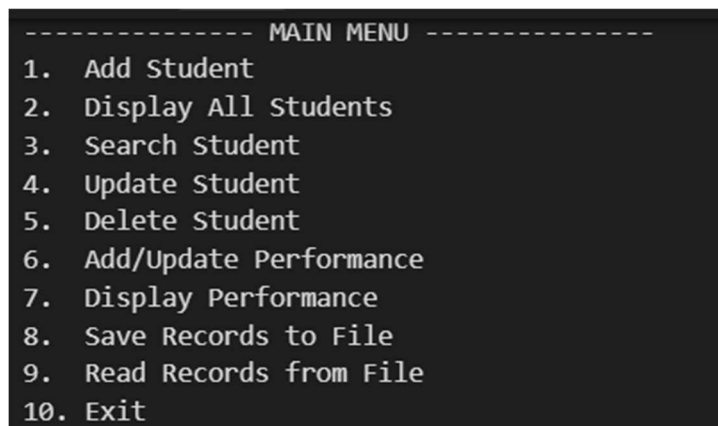
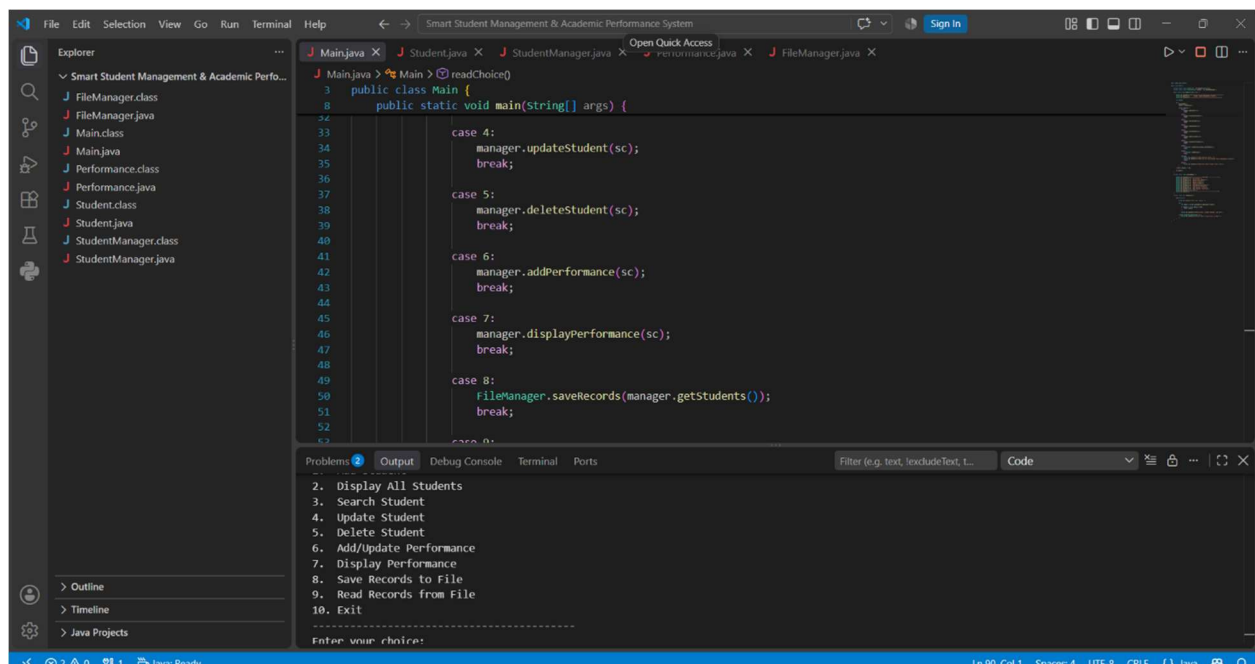
19. RESULTS AND SCREENSHOTS

The developed system successfully performs the major operations included in its scope.

The application was tested by adding student records, entering academic performance, searching and updating students, deleting records, and performing file operations.

The following screenshots should be added to the final report from the actual execution of the program.

to File



```
J Main.java X J Student.java X J StudentManager.java X J Performance.java X J FileManager.java
J Student.java > Student
1 public class Student {
2
3     private int studentId;
4     private String name;
5     private int age;
6     private String gender;
7     private String department;
8     private int semester;
9     private String email;
10    private String phone;
11
12    private Performance performance;
13
14    public Student(int studentId, String name, int age, String gender,
15                  String department, int semester,
16                  String email, String phone) {
17
18        this.studentId = studentId;
19        this.name = name;
20        this.age = age;
21        this.gender = gender;
22        this.department = department;
23        this.semester = semester;
```

OUTPUT:

FILE STORAGE / SAVE RESULT

----- MAIN MENU -----

1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Add/Update Performance
7. Display Performance
8. Save Records to File
9. Read Records from File
10. Exit

Enter your choice: 8

Records saved successfully in student_records.txt

The application creates:
student_records.txt

STUDENT RECORD MANAGEMENT SYSTEM – MAIN MENU / ADD S

```
=====
      STUDENT RECORD MANAGEMENT SYSTEM
=====
```

----- MAIN MENU -----

1. Add Student
2. Display All Students
3. Search Student
4. Update Student
5. Delete Student
6. Add/Update Performance
7. Display Performance
8. Save Records to File
9. Read Records from File
10. Exit

```
Enter your choice: 1
Enter Student ID: 101
Enter Name: Rahul
Enter Age: 20
Enter Gender: Male
Enter Department: CSE
Enter Semester: 3
Enter Email: rahul@example.com
Enter Phone: 9876543210
Student added successfully!
```

STUDENT PERFORMANCE REPORT

STUDENT PERFORMANCE REPORT

Student ID : 101
Name : Rahul
Department : CSE
Semester : 3

----- SUBJECT MARKS -----

Java : 85
Operating System : 90
Digital Logic : 82
Data Structures : 88
Computer Organization : 86

Average : 86.20%
Attendance : 82.0%
Grade : A
Performance : Very Good
Attendance : Satisfactory
=====

20. CHALLENGES FACED

During the development of the project, several practical challenges were encountered.

20.1 Managing Multiple Classes

One of the challenges was deciding how to divide the application into different classes. Keeping everything in a single class would make the program difficult to maintain. Therefore, responsibilities were separated into five classes.

20.2 Handling User Input

Console applications can encounter errors when users enter unexpected values. Exception handling and validation were added to prevent the program from terminating unexpectedly.

20.3 Connecting Student and Performance Information

Performance information needs to belong to the correct student. This was implemented by maintaining a Performance object inside the Student class.

20.4 File Handling

Saving and reading records required proper handling of Java I/O classes and possible IOException situations.

20.5 Maintaining Consistency

Since the classes interact with one another, method names, constructors, getters, setters, and object relationships had to remain consistent throughout the project.

21. LEARNINGS AND KEY TAKEAWAYS

Developing this project helped in understanding how individual Java concepts can be combined to create a complete application.

Object-Oriented Programming

The project provided practical experience in creating classes and objects and representing real-world entities using Java.

Encapsulation

Using private variables along with getters and setters helped demonstrate how data can be protected and accessed in a controlled way.

Collections

The use of ArrayList provided practical experience in managing a dynamic collection of student objects.

Exception Handling

The project demonstrated the importance of handling invalid user input and preventing unexpected program termination.

File Handling

Working with FileWriter, FileReader, and BufferedReader provided practical experience with storing and retrieving information from files.

Modular Programming

Separating functionality into different classes made the program easier to understand, test, and maintain.

Problem Solving

The project required thinking about the complete workflow of an application rather than implementing isolated Java programs.

22. PROJECT LIMITATIONS

The current implementation has several limitations.

1. The application is console-based.
2. It does not use a database.
3. Records are maintained in memory while the program is running.
4. The saved text file is used for viewing stored information but records are not automatically reconstructed into Student objects when the program starts.
5. There is no authentication mechanism.
6. The subjects are predefined in the Performance class.

These limitations are acceptable within the current project scope and can be addressed in future versions.

23. FUTURE ENHANCEMENTS

Several improvements can be made to the system in the future.

Database Integration

The current file-based storage can be replaced with a relational database such as MySQL using JDBC.

Graphical User Interface

A graphical interface can be developed using Java Swing or JavaFX.

Authentication

An administrator login system can be added to provide controlled access to student records.

Automatic Data Loading

The system can be enhanced to automatically load previously saved student records when the application starts.

Advanced Search

Search functionality can be extended to support:

- Student Name
- Department
- Semester
- Grade

Performance Analytics

Additional features could include:

- Subject-wise performance analysis
- Top-performing students
- Attendance statistics
- Semester-wise performance comparison

Report Generation

Student reports could be exported in formats such as PDF or CSV.

24. GITHUB REPOSITORY STRUCTURE

The project repository is organized as follows:

Student-Record-Management-System

Main.java

Student.java

StudentManager.java

Performance.java

FileManager.java

README.md

statement.md

The Java files contain the application source code, while README.md provides project documentation and statement.md contains the problem statement, project scope, target users, and high-level features.

25. REFERENCES

1. VIT Bhopal University, "VITyarthi – Build Your Own Project: General Project Instructions & Submission Guidelines."
2. Java Documentation and learning resources provided as part of the Java Programming course.
3. Course lecture notes and academic material related to Core Java and Object-Oriented Programming.
4. Java Development Kit (JDK) documentation.

26. CONCLUSION

The Student Record Management System is a simple but practical application developed using Core Java and Object-Oriented Programming concepts.

The system provides basic student management functionality along with academic performance and attendance management. It also demonstrates how student records can be saved and read using Java file-handling classes.

The project helped in understanding how concepts such as classes, objects, encapsulation, arrays, ArrayList, exception handling, strings, and file I/O work together in a real application.

Rather than focusing only on individual programming examples, the project provided an opportunity to think about application structure, user interaction, validation, data management, and modular design.

The current system provides a foundation that can be extended in the future with database connectivity, authentication, graphical interfaces, advanced analytics, and automated report generation.

Overall, the project successfully demonstrates the practical application of Core Java concepts to a real-world student record management problem.

Github Repository

<https://github.com/lalan25bai11005/Student-Record-Management-System>